

# Visitor, Command и Memento Design Patterns в C++

Семинар 15  
Серхан

Visitor Design Pattern - извличаме дадена операция от класовете в отделен клас (Visitor). Всеки клас приема Visitor-а чрез accept() и му казва кой е чрез this. Така добавяме нови операции без да пишем класовете.

Double Dispatch - с accept() разкриваме типа на обекта, с visitX() - типа на операцията; резултатът зависи от двата типа едновременно.

Command Design Pattern - капсулираме действие в обект. Позволява undo, опашка от действия и отложено изпълнение.

ICommand (интерфейс) - декларира undo(), redo(), execute().  
Конкретните команди го имплементират.

ConcreteCommand - конкретна команда, имплементира undo() и execute()

Receiver - обектът, върху който командата работи; бизнес логиката

Invoker - обектът, който задейства командите. Работи само с ICommand.

Memento Design Pattern - запазваме целия state на обекта преди всяко действие; undo - възстановява последния state.  
прости команди - само execute()

Visitor - отделя алгоритмите от обектите, върху които работят  
• Класове комуникират помежду си и искаме нови операции без да ги докосваме.

Пример: имаме различни роли (човек, инспектор, пожарник...),  
и различни сгради (дом, фабрика, банка...)  
Всеки човек прави различни неща в сградите.  
Искаме да добавяме нови роли и нови сгради без да докосваме класовете.

Решението: Visitor - не питаме сградата какво да прави,  
казваме кой сме и тя ни пуска



Сградите имат само accept(visitor) - казваме коя роля  
че чрез this.

class IVisitor { - интерфейс с метод за всеки тип сграда  
public:

virtual void visitHouse(const House& h) = 0

virtual void visitBank(const Bank& b) = 0

// още сгради

virtual ~IVisitor() = default;

};

class IBuilding {

public:

virtual void accept(IVisitor& visitor) const = 0;

virtual ~IBuilding() = default;

};

class House: public IBuilding {

public:

void accept(IVisitor& visitor) const override {

visitor → visitHouse(\*this);

↓  
e House&

class PersonVisitor: public IVisitor {

// override-ва методите на IVisitor

};

Double dispatch - две виртуални извиквания, всяко разкрива  
свои тип.

Command - капсулираме дадено действие в обект. Не викаш функцията директно, а създаваш команден обект, който носи в себе си всичко за изпълнение и отмяна на действие

Пример: Текстов редактор с undo

Document - Receiver - върху него се изпълнява действието

```
class Document {  
    string text;  
public:  
    void write { // имплементира се }  
    void erase(size_t count) { // ... }  
    void show() const { // .. }  
};
```

← Искаме да достъпваме и изпълняваме неговите методи

```
class ICommand {  
public:  
    virtual void execute()=0;  
    virtual void undo()=0;  
    virtual ~ICommand()=default;  
};
```

← интерфейс

```
class WriteCommand : public ICommand {  
    Document & doc;  
    string word;  
public:  
    // конструктор ...  
    void execute() override { doc.write(word); }  
    void undo() override { doc.erase(word.size); }  
};
```

← конкретна команда

← започваме какво сме написали - нужно за undo()



Memento Design Pattern - запазваме текущия state на обекта  
преди всяко действие в отделен обект.  
undo - възстановява последния state

• Използваме при сложни операции и undo е трудно изборим

Структура:

Originator - обектът, който state пази  
и има clone()

Caretaker - пази стековете от clone()-нати обекти, извиква  
undo/redo

ICommand - само execute()